

Tech Nation Visa - Exceptional Promise

Criteria: Optional Criteria 3 (Product & Engineering Impact)

Applicant: Odeyemi Olusegun Israel

Evidence 7: The AMTTP “War Room” Compliance Dashboard

1. Enterprise-Ready Product Interface

A core differentiator of the AMTTP platform is its bespoke compliance dashboard built with **Next.js 14 (App Router)**, **TypeScript**, and **Tailwind CSS**. Designed for R3–R6 enterprise users (Compliance Officers, Risk Analysts, and Super Administrators), the “War Room” provides real-time surveillance over every transaction flowing through the decentralised compliance engine.

This is not a prototype or wireframe. It is a fully functional, data-driven administrative interface consuming live API responses from the Python microservice layer, rendering interactive charts, alert feeds, and risk scoring panels.

War Room Tech Stack:

- **Framework:** Next.js 14 (App Router) + TypeScript + Tailwind CSS
- **Data Fetching:** TanStack React Query consuming 10+ Python FastAPI microservice endpoints
- **Charting:** Interactive risk score heatmaps, transaction volume timeseries, alert severity panels
- **Theme:** Dark mode (`bg-gray-950`) for extended surveillance sessions
- **Routing:** Server components by default; `'use client'` directive only for interactive panels

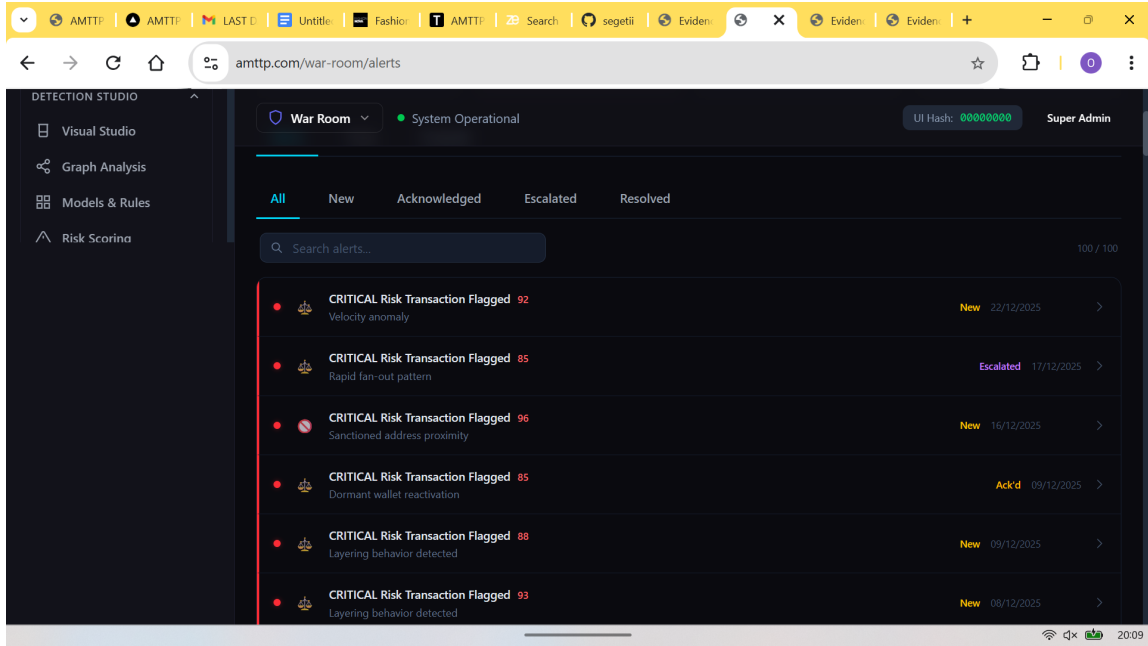


Figure 1: Live AMTTP War Room at amttp.com/war-room/alerts — 100 CRITICAL risk transaction alerts with severity scores, status tracking (New / Escalated / Acknowledged), and pattern classification (velocity anomaly, fan-out, sanctioned address proximity, layering behavior).

2. Role-Based Access Control (RBAC)

The dashboard enforces a six-tier Role-Based Access Control system (R1–R6), shared via a unified JSON configuration between the Flutter consumer app and the Next.js admin panel. This ensures that end users never see compliance-sensitive data, while senior administrators have full forensic capability.

RBAC Configuration (from `frontend/shared/rbac_config.json`):

Role	Label	Mode	Key Capabilities
R1	End User	Focus (Flutter)	Initiate own transactions only
R2	PEP End User	Focus (Flutter)	Enhanced monitoring, same UI
R3	Compliance Officer	War Room (Next.js)	View all transactions, reports
R4	Risk Analyst	War Room (Next.js)	Detection studio, policies
R5	Admin	War Room (Next.js)	Manage users, enforcement
R6	Super Admin	War Room (Next.js)	Emergency override, full access

```
// rbac_config.json (shared between Flutter & Next.js)
"R1_END_USER": { "mode": "FOCUS", "canViewOthersTx": false }
"R2_PEP_END_USER": { "mode": "FOCUS", "canViewOthersTx": false }
"R4_RISK_ANALYST": { "mode": "WAR_ROOM",
"canAccessDetectionStudio": true, "canEditPolicies": true }
"R6_SUPER_ADMIN": { "mode": "WAR_ROOM",
"canEmergencyOverride": true, "canManageUsers": true }
```

Figure 2: Six-tier RBAC system enforced via a single shared JSON configuration consumed by both the Flutter consumer app (R1–R2 Focus Mode) and the Next.js compliance dashboard (R3–R6 War Room Mode).

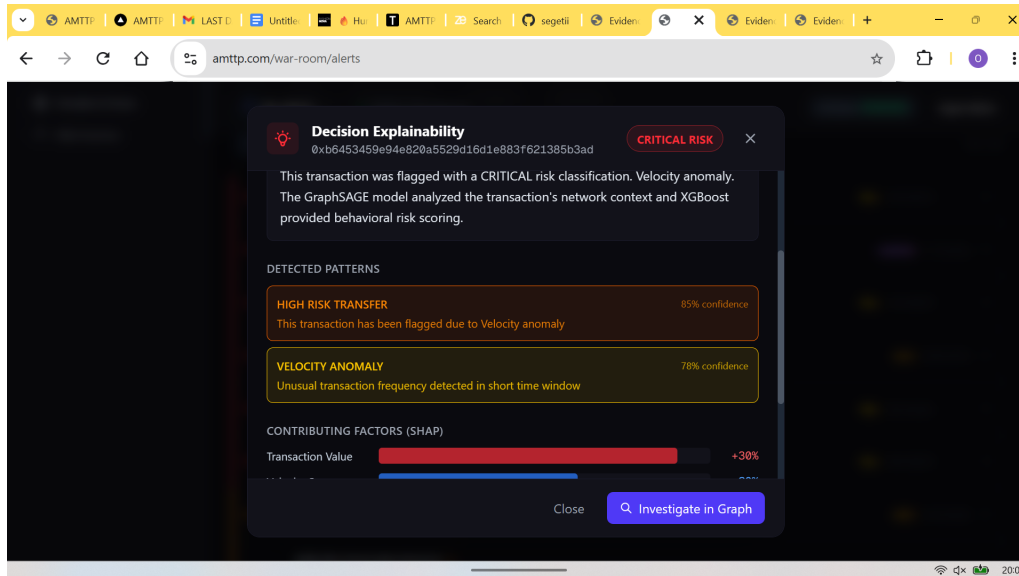


Figure 3: Decision Explainability panel — SHAP-based contributing factors for a CRITICAL risk alert. Shows detected patterns (High Risk Transfer 85%, Velocity Anomaly 78%) with an “Investigate in Graph” action linking to the graph analysis engine.

3. Technical Significance

Building a production-quality compliance dashboard from scratch — rather than using off-the-shelf tools like Retool or Grafana — demonstrates deep frontend engineering capability and a commitment to owning the full product stack. Every chart, table, and alert component was hand-coded by the applicant, as evidenced by the GitHub commit history (see Evidence 5).